

Article

Confidential Kubernetes Deployment Models: Architecture, Security, and Performance Trade-Offs

Eduardo Falcão ¹, Fernando Silva ², Carlos Pamplona ², Anderson Melo ², A S M Asadujjaman ² and Andrey Brito ^{2,*}

¹ Department of Computer Engineering and Automation, Federal University of Rio Grande do Norte, Natal 59078-970, Brazil; eduardo@dca.ufrn.br

² Department of Computers and Systems, Federal University of Campina Grande, Campina Grande 58190-970, Brazil

* Correspondence: andrey@computacao.ufcg.edu.br

Abstract

Cloud computing brings numerous advantages that can be leveraged through containerized workloads to deliver agile, dependable, and cost-effective microservices. However, the security of such cloud-based services depends on the assumption of trusting potentially vulnerable components, such as code installed on the host. The addition of confidential computing technology to the cloud computing landscape brings the possibility of stronger security guarantees by removing such assumptions. Nevertheless, the merger of containerization and confidential computing technologies creates a complex ecosystem. In this work, we show how Kubernetes workloads can be secured despite these challenges. In addition, we design, analyze, and evaluate five different Kubernetes deployment models using the infrastructure of three of the most popular cloud providers with CPUs from two major vendors. Our evaluation shows that performance can vary significantly across the possible deployment models while remaining similar across CPU vendors and cloud providers. Our security analysis highlights the trade-offs between different workload isolation levels, trusted computing base size, and measurement reproducibility. Through a comprehensive performance, security, and financial analysis, we identify the deployment models best suited to different scenarios.

Keywords: confidential cloud-native computing; Kubernetes; CVMs; AMD SEV-SNP; Intel TDX



Academic Editor: Jose María Alvarez Rodríguez

Received: 18 August 2025

Revised: 7 September 2025

Accepted: 12 September 2025

Published: 17 September 2025

Citation: Falcão, E.; Silva, F.; Pamplona, C.; Melo, A.; Asadujjaman, A.S.M.; Brito, A. Confidential Kubernetes Deployment Models: Architecture, Security, and Performance Trade-Offs. *Appl. Sci.* **2025**, *15*, 10160. <https://doi.org/10.3390/app151810160>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Cloud computing [1] allows businesses to harness the power of modern computing technology without having to deploy infrastructure that is costly to build and maintain. This outsourcing of computing to the cloud also brings the advantage of added redundancy and flexibility. A popular architecture for the deployment of outsourced applications is the microservices model [2], which allows splitting large applications into small independent workloads. These workloads are then containerized [3] and managed using orchestration tools, such as Kubernetes [4]. This approach of containerization and application splitting becomes more challenging to maintain and secure, but offers several benefits, including more efficient scaling opportunities and reduced impact from vulnerable microservices, as they cannot compromise other sub-components or their data.

However, the benefits of outsourcing containerized services to the cloud come with the concern of security, as many users now share the same infrastructure [5]. Cloud users now

must trust providers to protect their confidential data. In addition, bugs in the software used to build the cloud infrastructure can be exploited to compromise such an infrastructure. Thus, the data in the cloud is not only under threat from untrustworthy cloud operators but also from attackers.

To that end, confidential computing has recently emerged as a promising technology to protect the confidentiality and integrity of data in the cloud. Technologies under the umbrella of confidential computing are mainly of two types: those based on enclaves (e.g., Intel SGX) and those based on confidential virtual machines (e.g., AMD SEV-SNP). These technologies are used to transform containerized microservices into confidential workloads. However, these technologies introduce complexity, and merely deploying workloads with them does not automatically guarantee security [6].

In order to effectively secure workloads, confidential computing relies on process isolation and remote attestation. Process isolation is paramount for preventing the host and neighboring workloads from accessing confidential data. Remote attestation is essential for verifying that a remote process is executing in a Trusted Execution Environment (TEE) built on integrity-assured components. However, current implementations of confidential computing services make it difficult to perform such attestation. For example, a critical property for verifying the authenticity of a confidential workload is the initial measurement, which involves the UEFI firmware of VMs as well as other relevant pieces of software on the host. As the firmware resides on the cloud providers' premises and cloud users typically do not have access to it, it becomes challenging to pre-calculate the initial measurement and, hence, to properly perform remote attestation of workloads [6]. In addition to security, cost-effectiveness is a major concern when deploying workloads in the cloud. While the cost difference between confidential and non-confidential computing services may be minor, significant variation can exist between different deployment models for confidential workloads. Thus, careful consideration of trade-offs between different deployment configurations is necessary.

Existing works on confidential workloads mainly focus on individual container performance [7–11]. They do not fully address the challenges of deploying confidential cloud-native applications in a secure and cost-effective manner. In this work, we focus on the following research questions (RQ):

- **RQ1.** What practical deployment models exist for running confidential workloads in untrusted environments using existing technology?
- **RQ2.** What are the security levels provided in each deployment model?
- **RQ3.** What is the overhead in workload performance for the different deployment models?
- **RQ4.** What are the financial costs of using the different deployment models?

Our specific contributions are as follows:

1. We designed five different models for the deployment of confidential cloud native applications in Kubernetes and discussed their features (Section 4; addressing RQ1).
2. We performed a security analysis focusing on the isolation and measurement-reproducibility features of the different deployment models (Section 5.1; addressing RQ2).
3. We conducted a performance evaluation of workloads in different deployment models (Section 5.2; addressing RQ3). To that end, we deployed the models on platforms from three major cloud providers, utilizing the confidential computing solutions from two of the most popular CPU vendors.
4. We performed a financial cost analysis of hosting a varying number of workloads in different deployment models (Section 5.3; addressing RQ4).

The rest of this paper is organized as follows. Section 2 describes technologies relevant to understanding this work. Section 3 presents our threat model. Section 4 details the design of our deployment models. In Section 5, we present the security analysis, the performance evaluation, and the financial analysis of the designed deployment models. Finally, Sections 6 and 7 present related work and closing remarks, respectively.

2. Background

In this section, we present confidential computing technologies that enable the creation of Confidential VMs. Then, we discuss different technologies that enable confidential Kubernetes clusters. Finally, we detail how remote attestation works for different technologies.

2.1. Confidential Computing

Confidential computing is a technology conceived primarily to protect data while it is being processed, particularly in cloud environments. It achieves this protection through Trusted Execution Environments (TEEs), which provide isolated and secure enclaves where sensitive data and code can be executed without exposure to the host system or unauthorized entities. TEEs ensure that data remains encrypted and inaccessible to the underlying operating system and hypervisor, thereby enhancing privacy and security in untrusted environments.

There are two primary types of TEEs, namely, enclave-based and CVM-based. Enclave-based TEEs, such as Intel SGX, operate at the process level, isolating specific applications within a secure enclave. This approach offers fine-grained protection but requires applications to be explicitly designed or modified to leverage enclaves, which can introduce development complexity. In contrast, CVM-based TEEs, such as **Intel TDX and AMD SEV-SNP**, extend security to entire virtual machines, providing a more flexible and transparent solution that does not require application modifications. While CVMs offer broader compatibility and ease of adoption, they tend to involve a larger Trusted Computing Base (TCB), which increases the system's overall attack surface. The choice between enclave-based and CVM-based TEEs depends on the specific security, performance, and integration requirements of a given workload.

The **Intel TDX** technology was introduced in 2023. Unlike SGX, which protects only a single application, TDX secures an entire VM through a concept known as trust domains (TDs). This protection is achieved through two primary mechanisms, namely, isolation and cryptographic techniques. To enable isolation, Intel introduced a new CPU mode called **SEcure Arbitration Mode (SEAM)**. This mode has two major security features, as follows: (1) private keys used to protect TDs can only be accessed within this mode, and (2) it secures a memory region known as **SEAM-Range**, which can only be accessed by software running in the same region [12]. The Intel TDX module, hosted in this secure region, runs in SEAM mode and acts as a micro-hypervisor, managing TD VMs independently of traditional hypervisors [13]. As a result, hypervisors no longer directly create VMs but instead must request the TDX module to do so, ensuring better isolation and integrity protection.

An interesting feature provided by Intel TDX is TD partitioning [14]. It enables nested virtualization within TD, allowing a primary VM (L1) to function as a hypervisor for secondary VMs (L2) while maintaining hardware-enforced isolation. This capability could support secure multi-tenant deployments, such as Kubernetes clusters in which the control plane runs at L1 and workloads operate in isolated L2 instances.

The **AMD Secure Encrypted Virtualization Secure Nested Paging (SEV-SNP)** technology enhances VM security by ensuring both memory confidentiality and integrity. Confidentiality is achieved through AES-based encryption, where encryption keys are provided at the hardware level and remain exclusive to each VM, preventing unauthorized

access. Memory integrity is enforced using a structure called the Reverse Map Table (RMP), which tracks ownership (hypervisor or VM) for each memory page and prevents direct manipulation by software, thereby mitigating potential security breaches [15].

In addition to memory encryption and integrity control, the AMD Secure Processor (AMD-SP) measures all the memory pages injected during the creation of the CVMs. This measurement enables the production of attestation reports, which include a hash of the VM construction process and relevant configurations of the physical processor (e.g., firmware version, hyper-threading, debug mode). Later, processes inside the CVMs can request attestation reports from the AMD-SP, and third parties can then use these reports to remotely attest that a VM is running in the AMD SEV-SNP confidential mode.

Other TEEs are also emerging as part of the evolving confidential computing landscape. Examples include IBM Power10's Protected Execution Facility [16] (targeting the Power Instruction Set Architecture), Arm's Confidential Compute Architecture [17] (designed for Armv9-A architecture), and Hygon Cloud Secure Virtualization [18] (a TEE for x86 CPUs from Hygon, supported by Alibaba Cloud). These solutions share the core objective of protecting data in use via hardware-enforced isolation and attestation but differ in their target platforms, protection granularity, and ecosystem maturity. In this work, we focus on Intel TDX and AMD SEV-SNP, as they are the most widely supported confidential computing technologies by major cloud service providers (CSPs).

In summary, Intel's and AMD's CVMs share many similarities inherited from the confidential computing model. Both rely on an attestation report with relevant measurements used for integrity verification. The security of such a report is guaranteed in both technologies because it is signed by a key internal to the processor. However, AMD SEV-SNP lacks runtime measurement registers (RTMRs)—a feature present in Intel TDX—and requires integration with technologies such as the virtual Trusted Platform Module (vTPM) if runtime measurement is necessary. Among the three major cloud providers—AWS, GCP, and Azure—only AWS and GCP offer support for computing initial measurements offline. AWS provides both the VM firmware code and instructions to pre-compute the launch measurement for its VMs supporting AMD SEV-SNP. GCP offers guidelines for verifying AMD SEV-SNP and Intel TDX measurements from firmware binaries, but does not disclose the firmware code.

2.2. Confidential Kubernetes

Kubernetes is one of the most popular container orchestration tools and is widely used to efficiently manage and scale containerized workloads. However, its infrastructure is susceptible to various security threats, including vulnerabilities in the underlying hardware and risks of unauthorized access to sensitive data processed by workloads. To mitigate these risks, one strategy is to run Kubernetes clusters confidentially by leveraging confidential computing technologies, particularly Confidential VMs. Additionally, some frameworks have been designed to streamline the integration of Kubernetes workloads with confidential computing. Next, we explore these frameworks in detail.

2.2.1. Kata Containers, Confidential Containers, and Peer Pods

Kata Containers [19] is a container runtime that enhances security by running containers inside lightweight VMs. Its main goal is to achieve strong isolation by running workloads in VMs rather than in traditional containers, which rely on Linux namespaces and cgroups for isolation. Still, it is compatible with regular container runtimes (e.g., containerd), ensuring seamless integration with Kubernetes. This makes Kata Containers particularly well-suited for multi-tenant environments, such as public clouds, where it

helps protect workloads from each other. For instance, container escape attacks would be rendered useless since there are no other workloads running on the same VM.

In the context of Kubernetes, Kata Containers relies on two components—shim-v2 and the kata-agent. This architecture is illustrated in Figure 1.

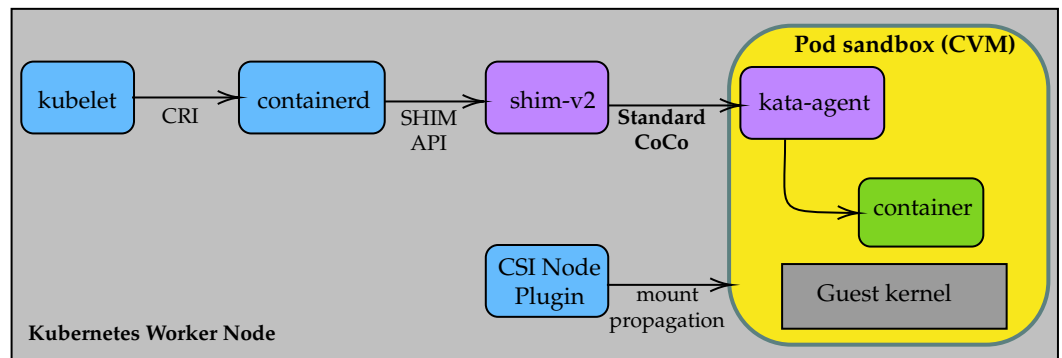


Figure 1. Kata Containers architecture. The CoCo extension adds TEE support to PodVMs. In this setting, PodVMs are launched by the node’s runtime [20].

The shim-v2 component relies on QEMU/KVM to create a VM for each pod, and implements a containerd Runtime v2 (Shim API), enabling the creation of OCI-compatible containers (Open Container Initiative (OCI) is a standard for lightweight and portable containers that run consistently across environments). The container process is spawned by the kata-agent, a daemon process running inside the VM, which interfaces with shim-v2 to receive and execute container management commands.

Kata Containers was announced by Intel in 2017. Over time, with the rise of confidential computing, the **Confidential Containers (CoCo)** [21] project was created. CoCo is an open-source Cloud Native Computing Foundation (CNCF) project that aims to integrate confidential computing with cloud-native workloads. Since Kata Containers already uses VMs for running workloads, CoCo leverages the Kata Containers runtime and extends it to support Confidential VMs. In essence, CoCo enables pods to run as Confidential VMs and introduces components at the CVM level for the remote attestation of these VMs.

Finally, there is the **Peer Pods** framework (Figure 2), which extends Kata Containers to run sandboxed containers outside the Kubernetes workers, either on bare metal or in public clouds. This capability is particularly useful for (i) enforcing stronger tenant isolation by running workloads in separate VMs and (ii) fine-grained scalability at the workload level, instead of at the worker-node level—particularly advantageous for burst workloads, ephemeral jobs, or low-duty-cycle tasks.

The Peer Pods framework is mainly enabled by adding the Cloud API Adaptor [22], which provides support for remote hypervisors. It runs as a DaemonSet on each Kubernetes node and is responsible for receiving commands from the shim-v2 process and implementing them for Peer Pods. When it receives commands that are related to the pod lifecycle, it will use IaaS APIs (from the cloud service provider or locally through libvirt) to create and delete the Peer Pod VMs. Additionally, it facilitates communication between shim-v2 and the kata-agent inside the pod VM, ensuring seamless workload integration in remote environments.

Due to its architecture, volume provisioning for Peer Pods differs from that of standard pods. The CoCo project extends the Container Storage Interface (CSI) to support plugins tailored for Peer Pods. At the cluster level, the CSI Controller Plugin, running in the control plane, manages the lifecycle of block device volumes – handling their creation, deletion, and determining to which node they should be attached. In the Peer Pods architecture, the CoCo CSI Controller Plugin intercepts volume-attach requests, defers them until the

corresponding pod is created, and then stores the volume-attachment status in a CSI Custom Resource Definition (CRD) associated with the Peer Pod. At the node level, the CSI Node Plugin is responsible for mounting the volume and exposing it to the container runtime. In the case of Peer Pods, the CoCo CSI Node Plugin runs inside the Peer Pod CVM. It continuously monitors the status of the Peer Pod's CSI CRD, and once the volume is confirmed as attached, it mounts the volume inside the CVM and makes it available to the container.

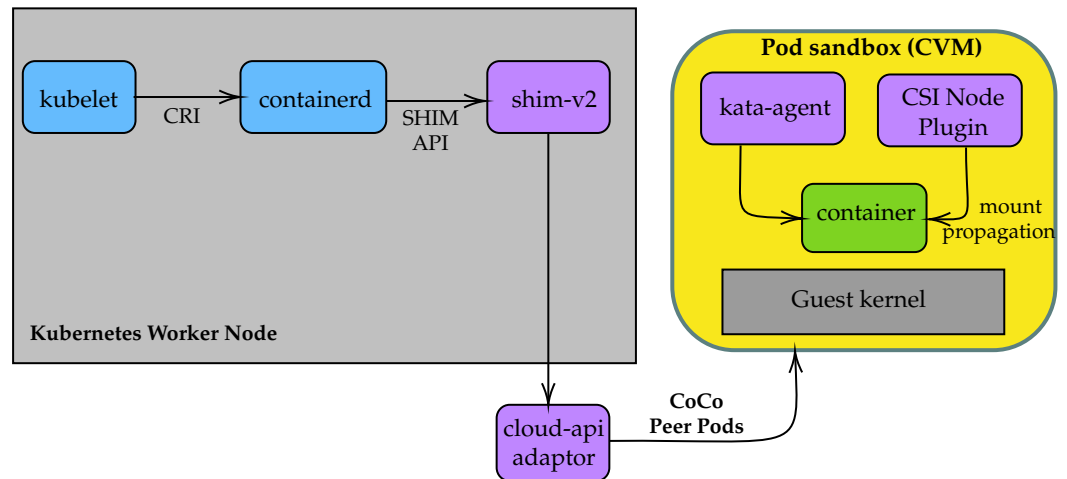


Figure 2. Kata Containers Peer Pods architecture, where Confidential workloads are provisioned externally via a Cloud API Adaptor, isolating them from the Kubernetes worker node [20].

Figures 1 and 2 illustrate this mechanism for standard CoCo and CoCo Peer Pods, respectively. Notice that in the former (CoCo), the CSI Node Plugin runs on the Kubernetes worker node, while in the latter (Peer Pods), it resides within the CVM, inside the pod itself. Thus, when using CoCo, the Kubernetes operator must configure a DaemonSet with the CoCo CSI Node Plugin so that each worker node has the ability to provision volumes. For CoCo Peer Pods, the developer must use CSP images that implement the CoCo CSI Node Plugin and configure sidecar containers to perform volume-management operations.

2.2.2. SPIFFE and SPIRE

Confidential Containers and Peer Pods are frameworks designed to facilitate the deployment and attestation of confidential, cloud-native workloads within Kubernetes clusters. In a broader context of node attestation, there is the SPIFFE standard (Secure Production Identity Framework for Everyone) [23], which can be used to verify the integrity of nodes running with different technologies for the purpose of issuing software identities. Thus, if used to verify the integrity of Kubernetes worker nodes running inside CVMs, SPIFFE, along with its reference implementation SPIRE, can support confidential cloud-native applications in Kubernetes.

SPIFFE replaces traditional credential provisioning methods, such as environment variables or Kubernetes secrets, which are often stored unencrypted, with strong, automatically issued, cryptographically verifiable identities. These identities are backed by attestation and signed by a trusted certificate authority, replacing the standard strategy in which protecting one secret depends on another. By doing so, SPIFFE eliminates the need for long-lived, static credentials and significantly improves the security posture of heterogeneous infrastructures. SPIFFE is a graduated project from CNCF, and is considered mature for production usage—both SPIFFE and its reference implementation, SPIRE, are graduated projects from the Cloud Native Computing Foundation (<https://www.cncf.io/projects/>, accessed on 13 September 2025). Furthermore, continuous credential verification is also

recommended by agencies such as NIST for establishing trust in workloads in a zero-trust architecture [24].

SPIRE (the SPIFFE Runtime Environment) [25] adopts an agent–server architecture in which the SPIRE server acts as the root of trust, issuing identities for nodes and workloads. Identities are only issued after trust is established; nodes must first pass a remote attestation process with the server, and workloads must undergo local attestation by the SPIRE agent. This architecture makes SPIRE particularly suitable for confidential Kubernetes workloads, where node integrity verification is critical to ensure a workload is executing in a protected environment. By leveraging node attester plugins capable of verifying CVMs running on technologies such as Intel TDX or AMD SEV-SNP, SPIRE can ensure that some identities are only granted to confidential workloads.

In the next section, we detail the remote attestation workflow for Confidential Containers and SPIRE.

2.3. Remote Attestation Mechanisms

Before a confidential workload can access sensitive data, it must first undergo remote attestation. Remote attestation is a mechanism by which a peer, known as the Attester, produces verifiable evidence about its own state, enabling a remote peer, the Relying Party, to assess its trustworthiness. This process is supported by a critical third entity, the Verifier, which evaluates the evidence using predefined appraisal policies and generates attestation results to assist the Relying Party’s decision-making process. The Verifier’s evaluation is informed by trusted reference values supplied by another key entity in the architecture, the Endorser. This architecture is formalized in IETF RFC 9334 (Remote Attestation procedureS—RATS) [26].

In the context of confidential computing, this process relies on TEE-generated attestation reports to verify the security properties of the TCB, including firmware versions, component measurements, certificate signatures, and the overall chain of trust. Confidential Containers utilizes the Trustee framework [27] to validate these attestations and conditionally release secrets only to verified workloads. Alternatively, SPIRE can be employed with CVM Node Attestors to establish trust in the integrity of nodes and to issue identities that can be leveraged to securely retrieve sensitive data.

2.3.1. CoCo Remote Attestation

The Trustee, located in the Trusted Server, consists primarily of the following three components: the Key Broker Service (KBS), the Attestation Service (AS), and the Reference Value Provider Service (RVPS). According to the RATS architecture, the Attestation Service acts as the Verifier, the Reference Value Provider Service acts as the Endorser, and the Key Broker Service functions as the Relying Party, making trust-based decisions based on attestation results. On the guest side, the CVM includes the Confidential Data Hub (CDH) and the Attestation Agent (AA), which collectively represent the Attester in RATS terminology. Such architecture is presented in Figure 3.

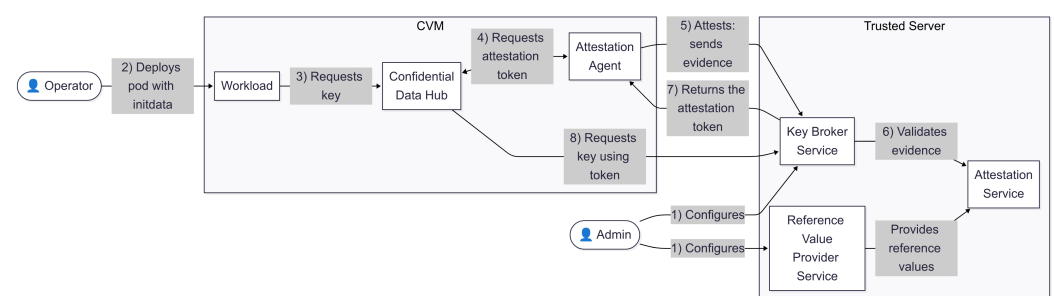


Figure 3. Remote Attestation workflow for CoCo [28].

The workflow proceeds as follows. First, an admin configures the KBS with keys and proper attestation policies. These policies are expressed using the Open Policy Agent (OPA) standard and the Rego language, and can specify, for instance, that a key should only be released to AMD SEV-SNP VMs with a valid launch measurement. The admin must also specify the reference values for the properties in the RVPS, such as valid AMD SEV-SNP and Intel TDX measurements. These values are used to appraise evidence later during the attestation process.

When the operator starts a workload, it passes some information in the manifest through initdata (step 2), including the KBS address, OPA policies for the Kata Agent execution controller, and certificates. For instance, a Kata Agent policy can forbid the execution of additional processes in the pod, providing higher isolation. Such information is leveraged to create evidence for remote attestation of different TEEs, such as AMD SEV-SNP and Intel TDX.

When the CVM starts, the workload requests a symmetric encryption key from the CDH (step 3). Then, the CDH requests an attestation token from the AA (step 4) so that it can request a key from the KBS. The AA initiates a challenge–response protocol with the KBS to produce and validate the evidence (step 5). The KBS issues a nonce, and the AA responds with a signed attestation report that includes this nonce. The KBS forwards the evidence to the AS (step 6), which evaluates it against the reference values and policy constraints in the RVPS. Based on the attestation result returned by the AS, the KBS decides whether the CVM is authorized to access the requested key. If approved, the KBS returns the attestation token to the AA (step 7), which passes it back to the CDH. Finally, the CDH requests a key to the KBS and sends it to the workload (step 8). With the key, the workload can access encrypted resources, such as secure volumes managed by the CDH.

2.3.2. SPIRE Remote Attestation

First, let us introduce the basic concepts of SPIFFE and SPIRE. A typical SPIRE deployment consists of two main components, namely, the SPIRE server and the SPIRE agent. The SPIRE server is responsible for attesting nodes, and after that, SPIRE agents will attest workloads. Once their trustworthiness is verified, the server issues them identities known as SVIDs (SPIFFE Verifiable Identity Documents), typically in the form of X.509 certificates. Each identity is associated with a unique SPIFFE ID, such as *spiffe://example.com/db-server*, which serves as a strong, cryptographically verifiable name for the workload or node. The assignment of SPIFFE IDs is governed by registration entries, which define the mapping between a SPIFFE ID and a set of selectors—attributes derived during attestation. These registration entries are essential for both node and workload attestation, as they determine which SVIDs are issued to which entities.

Now, let us examine how these components align with the RATS architecture. In this context, the SPIRE server acts as the Verifier, while Registration Entries serve as the Endorsers. Although SPIRE manages identities, encryption and decryption keys are typically provided by external services. Therefore, the Key Management Service (KMS) functions as the Relying Party. Finally, the CVM includes the SPIRE agent, which assumes the role of the Attester in RATS terminology. This architecture is illustrated in Figure 4.

The workflow proceeds as follows. First, an operator configures the KMS with encryption/decryption keys and corresponding SPIFFE IDs authorized to retrieve each key. These are simple key-value tuples, where the SPIFFE ID is the key of the tuple, and the encryption/decryption key is the value of the tuple. Furthermore, the operator must specify the registration entries, such as the valid AMD SEV-SNP and Intel TDX measurements. The registration entry maps a set of selectors to an SPIFFE ID. These values are used by node and workload attestors to decide which SPIFFE ID a node/workload should receive.

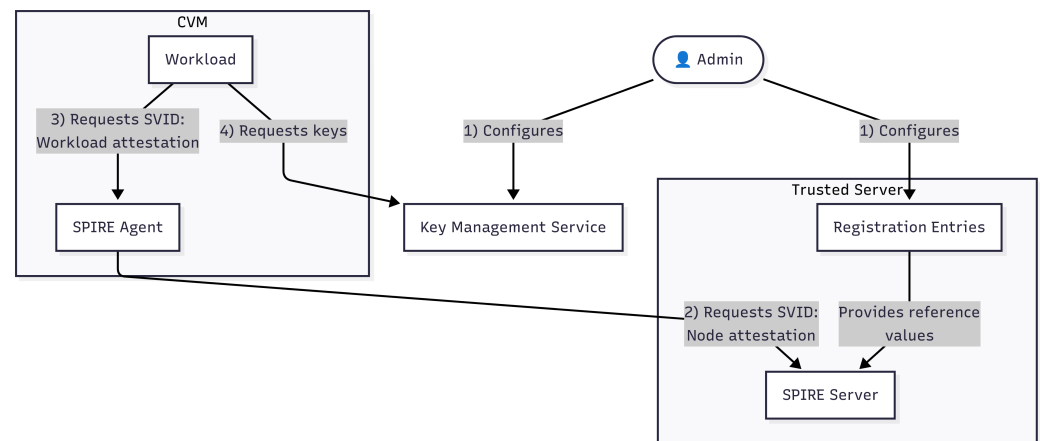


Figure 4. Remote Attestation workflow for SPIRE.

When the CVM starts, the agent requests its SVID from the server. During this process (step 2), the agent engages in a challenge–response protocol with the server; the server issues a nonce, and the agent responds with a signed attestation report containing it. The server then evaluates the report against the registration entries to determine which SPIFFE ID to assign to the node. Upon successful attestation, the node receives its SVID, private key, and a list of workload registration entries bound to its SPIFFE ID. With these in place, the agent can proceed with workload attestation and issue SVIDs to workloads (step 3). Finally, now in possession of an SVID, the workload can fetch its encryption/decryption key from the KMS (step 4).

SPIRE is an open-source project [25] that follows a plugin-oriented architecture. Servers can be configured with multiple node attestors, since they are used to attest multiple agents, and each agent is configured with a single node attestor. Currently, there are node attestors that verify information from VMs of public clouds (Amazon Web Services—AWS, Azure, and Google Cloud Platform—GCP). In previous works [29,30], we created the AMD SEV-SNP Node Attestor and the Intel TDX Node Attestor, plugins that verify whether a VM is running confidentially with the AMD SEV-SNP or Intel TDX technology. Furthermore, Pontes et al. (2024) [31] proposed the Hybrid Node Attestor, a plugin that lets one configure multiple node attestors on the agent side. In this work, we leverage such node attestors and combine them with other cloud tools to propose different Confidential K8s deployments.

3. Threat Model

The main critical components of our threat model are as follows: (i) the CoCo and SPIRE infrastructures, (ii) the CVM stack and its TCB, and (iii) the Kubernetes toolset. Below, we outline the security assumptions for each of these components.

Regardless of relying on CoCo or SPIRE, a trustworthy component must serve as the root of trust to attest to other components. In Section 2.3 (remote attestation), we call this component the Trusted Server. In a CoCo infrastructure, this role is played by the Trustee, which includes the Key Broker Service, Attestation Service, and Reference Value Provider Service. In a SPIRE-based setup, the SPIRE server serves this role. Establishing trust in the Trustee or SPIRE server can rely on the same attestation mechanisms described earlier, on an ad hoc attestation process, or on deployment in a trusted on-premises environment where the user is responsible for validation. We assume that the user either attests the Trustee or SPIRE server, or implicitly trusts it. Conversely, the attestation agent is assumed to run in an untrusted environment and becomes trusted only after successfully passing attestation.

Since, in this work, we consider the attestation agent runs in CVMs, we consider all attacks within the scope of AMD SEV-SNP and Intel TDX. For instance, an attacker

may exploit vulnerabilities in the hypervisor or compromise other components of the hosting environment, such as the host OS, device drivers, or BIOS. As with TEEs, such as AMD SEV-SNP and Intel TDX, we also account for privileged adversaries, including cloud administrators, who may attempt various attacks. These could involve accessing unencrypted data stored on disk or transmitted through I/O, as well as deploying a VM with a modified image containing malicious code. The CVM trusts only the CPU hardware and vendor-signed software modules, which include the Authenticated Code Modules and the Intel TDX module, and the AMD SEV-SNP firmware.

Finally, in all proposed Kubernetes deployment models, workloads (pods) execute within Confidential VMs. These workloads may be malicious and attempt to steal information from other workloads, especially the identities. The master node operates outside CVMs, while worker nodes (if present) run within them. In all cases, we assume the Kubernetes toolset may contain bugs. While such bugs can affect the availability and integrity of the cluster, some deployment models enforce workload isolation that helps mitigate integrity risks. For instance, cryptographic measurements that include critical information, such as the Trusted Server endpoint, can be used to defend against man-in-the-middle and impersonation attacks.

The next section outlines the different confidential Kubernetes deployment models.

4. Confidential Kubernetes Deployment Models

Confidential Kubernetes deployments fall into two models: confidential node-driven and confidential workload-driven. In the node-driven model, worker nodes run within Confidential VMs, while in the workload-driven model, only pods do, with no worker nodes. In both deployments, the master node may run in a regular VM or in on-premises infrastructure. These deployment models have a direct impact on key system features, including resource consumption, workload isolation, trusted computing base (TCB) size, infrastructure control (affecting measurement reproducibility), support for multiple TEEs in a single cluster, and cloud portability.

Next, we depict each deployment and its corresponding features and drawbacks.

4.1. Confidential Node-Driven Kubernetes

In this model, confidentiality is achieved by running worker nodes within Confidential VMs and attesting them with SPIRE. Confidential node-driven Kubernetes can be deployed either directly on CVMs or using nested VMs that support confidential computing. Figure 5 illustrates confidential node-driven deployments with and without the nested VMs feature.

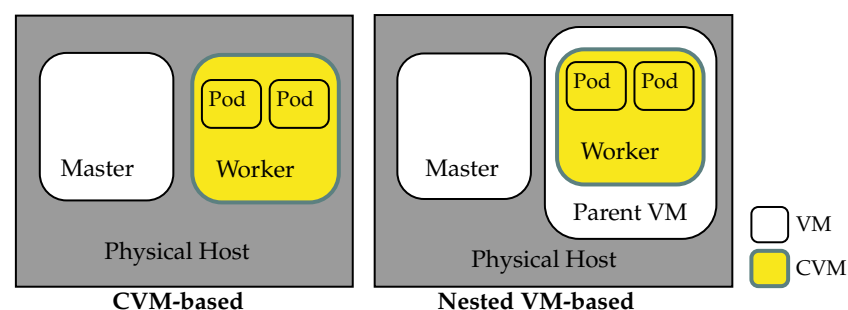


Figure 5. Confidential node-driven deployments.

Confidential node-driven deployments optimize resource consumption by running entire nodes within Confidential VMs, where pods share the same kernel and underlying system resources. However, this comes at the cost of reduced workload isolation, as all pods within a worker node operate within the same Confidential VM. One drawback of such an approach is that larger TCBs increase the attack surface and the complexity of

components' verification—a small vulnerability in one component can compromise the entire system. On the other hand, this approach is highly portable across cloud service providers (CSPs) that support confidential computing, allowing organizations to deploy Kubernetes clusters on different infrastructures with minimal modifications.

Another drawback of CVM-based deployments is that the reproducibility of measurements depends on the CSP's implementation, particularly the availability of UEFI code and well-documented guidelines for measurement verification. In contrast, when using nested VMs, the parent VM functions as the hypervisor for the child Confidential VMs, giving operators full control over the virtualization stack. This enables CVMs' measurement reproduction, as the entire environment can be customized and audited.

Finally, CVM and nested VM-based deployments enable multi-TEE environments by instantiating worker nodes with flavors supporting different TEE technologies.

4.2. Confidential Workload-Driven Kubernetes

Most of the confidential workload-driven Kubernetes deployments rely on CoCo to execute each workload in a separate Confidential VM (or PodVM, in CoCo terminology)—it is also possible, although less common, to deploy workload-driven Kubernetes with SPIRE, as detailed in Section 5. The key distinction between workload-driven deployments lies in the use of nested VMs or Peer Pods, each providing distinct features and capabilities. Figure 6 illustrates confidential workload-driven deployments with and without the nested VMs and Peer Pods features.

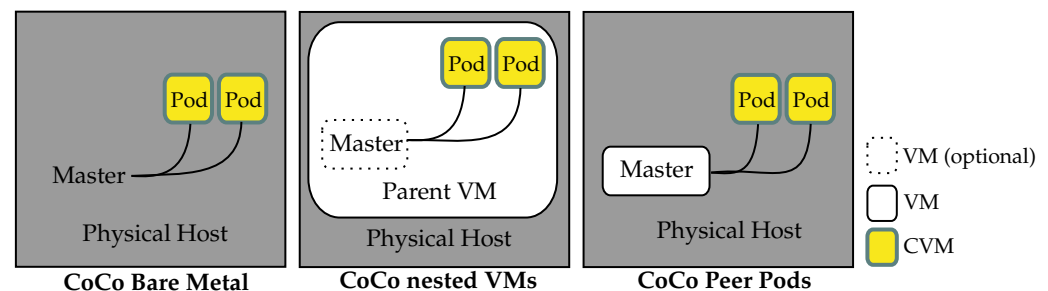


Figure 6. Confidential workload-driven deployments. For CoCo nested VMs, a master can also be on a separate VM (or in multiple, in case of replication).

The first model, CoCo Bare Metal, is well suited for on-premises infrastructures (or bare-metal providers), as it requires control over the hypervisor to deploy the Kubernetes master at this level. Alternatively, the CoCo nested VMs model enables a similar deployment in the cloud, with the parent VM acting as the hypervisor. This approach combines the benefits of on-premises control, such as full-stack management and reproducible measurements, with the scalability and convenience of public cloud computing, eliminating the need to acquire and maintain physical servers.

Finally, there is the CoCo Peer Pods approach, an alternative to scenarios where the platform does not support nested VMs but PodVM isolation is required. A drawback of the CoCo Peer Pods lies in the control over resource allocation. For CoCo Peer Pods, such a control is limited since it depends on CSP-defined instance sizes for confidential machines, typically starting at 2 vCPUs and 8 GB RAM. In contrast, the other workload-driven and node-driven deployments enable fine-grained resource customization, allowing smaller instances (e.g., 1 vCPU and 4 GB RAM) through pods' resource configuration.

4.3. Summary

In summary, there are six features we can assess in terms of the different deployments: resource-consumption efficiency, the possibility of configuring pod size, workload isolation, TCB size, measurement reproducibility, and cloud portability. A deployment is considered

highly resource-efficient when workloads share the same kernel, which results in low workload isolation. Conversely, deployments with low resource efficiency allocate each workload to an individual kernel, thereby providing strong workload isolation. Table 1 provides a comparative characterization of these deployments based on the listed features.

Table 1. Features of the different K8s deployments.

Deployment	Resource Efficiency	Pod Size Config.	Workload Isolation	TCB Size	Measurement Reproduc.	Cloud Portability
CVM	high	user-defined	low	worker node (large)	with gaps	yes
Nested VM	high	user-defined	low	worker node (large)	complete	no
CoCo Bare Metal	low	user-defined	high	PodVM (small)	complete	yes
CoCo nested VMs	low	user-defined	high	PodVM (small)	complete	no
CoCo Peer Pods	low	CSP-defined	high	PodVM (small)	with gaps	yes

The choice of deployment strategy depends on available resources and main objectives such as security, resource efficiency, and portability. A practical way to analyze this decision is by evaluating the drawbacks and constraints of each deployment option.

- If optimizing **resource consumption is a top priority**, node-driven deployments are the preferred choice, as pods share the underlying kernel and can be precisely sized for efficiency. A potential scenario is a cluster serving trusted workloads from a single user (or multiple users that fully trust each other's workloads).
- For **strong workload isolation**, a workload-driven deployment (one of the CoCo strategies) is ideal. Such deployments are well-suited for offering Kubernetes as a platform-as-a-service (PaaS), where workloads from multiple users or organizations run side by side.
- If **measurement reproducibility is essential (i.e., completely removing the provider from the TCB)**, CoCo Bare Metal or one of the nested VM deployments is the best choice. Furthermore, opting for deployments with smaller TCBs favors measurement stability and decreases the attack surface. This is an important feature for critical workloads executing in untrusted clouds.
- In situations where **portability is a priority**, nested VM approaches should be avoided, as they are currently supported only on Azure. Furthermore, CoCo favors portability among CSPs.
- All deployments support workloads running with **different TEEs within a single cluster**. This is particularly interesting because, although Intel TDX and AMD SEV-SNP are quite similar, each has its own strengths and limitations, making them better suited for different application requirements.
- Finally, the node-driven deployments protect the whole cluster; therefore, they are able to process unmodified applications (assuming they follow DevOps best practices, such as signed images). For the workload-driven deployments, the application manifest can be completely inspected and modified in its path through Kubernetes (e.g., having images or configurations changed, secrets exposed, containers injected); thus, it is important to rethink the deployment, especially ensuring how secrets and sensitive configurations are delivered, and using attestation to allow only attested workloads to interact.

The decision-making process is illustrated in the decision tree shown in Figure 7.

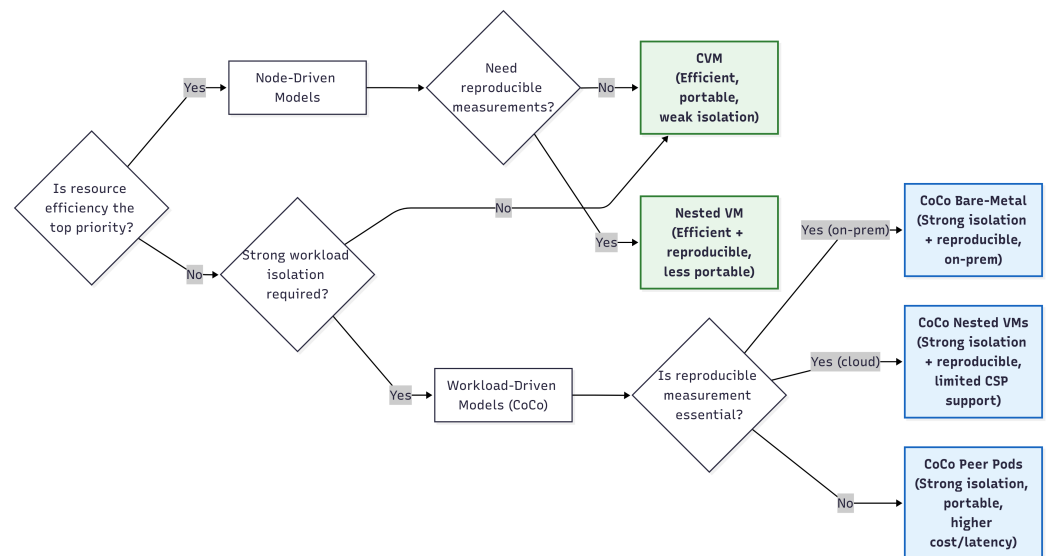


Figure 7. Decision tree summarizing the choice of deployment model according to the requirements.

5. Evaluation

In this section, we evaluate the security, performance, and financial costs of different confidential Kubernetes deployments. Our security analysis focuses on two key aspects, namely, isolation between workloads and reproducibility of CVM measurements. For the performance evaluation, we conducted benchmarking experiments to assess CPU, RAM, and I/O performance across the different deployment configurations. Finally, we analyze the financial costs associated with different deployment strategies by comparing the expenses incurred when running varying numbers of workloads. This cost assessment provides insights into the trade-offs among security, performance, and economic feasibility in different confidential Kubernetes deployments.

5.1. Security Analysis

The confidential Kubernetes clusters in this work leverage CoCo and SPIRE to verify the integrity of CVMs. Figure 8 presents the placement of the SPIRE/CoCo agent in workload-driven and node-driven deployments. Recall that the SPIRE server or Trustee must be deployed in a trusted on-premises installation or within a CVM that has been properly attested.

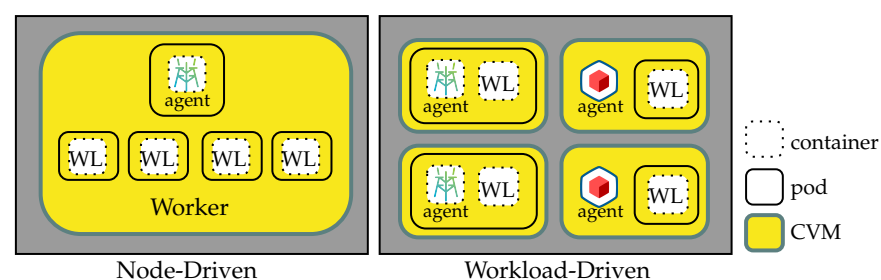


Figure 8. Deployment of CoCo agent and SPIRE agent in different confidential K8s models.

Isolation between workloads enhances the security of deployments. In node-driven deployments, the worker node runs within a CVM, while the SPIRE agent operates as a pod within the Kubernetes worker. The agent exposes its socket for workload attestation via shared volumes, resulting in a low level of isolation since multiple workloads rely on the same agent. In this setup, a malicious workload could attempt to impersonate others to obtain SVIDs fraudulently. To mitigate this risk, workload plugins must be carefully selected, and robust selectors should be used to define secure registration entries.

In contrast, workload-driven deployments provide a higher level of isolation by ensuring that workloads do not share the same CVM. Here, the SPIRE agent runs as a sidecar within each workload's pod, meaning each workload has its own dedicated agent. By running the agent inside the pod, CoCo achieves an even higher level of isolation. Both setups make it harder for impersonation attacks to succeed, since it is possible to deploy agents in CVMs with different measurements and use this property to restrict the set of sensitive information available to each node. As a result, this deployment model is well-suited, for example, for scenarios such as platform-as-a-service environments, where multiple users and organizations require strong workload isolation without compromising security.

Another key security advantage is the ability to compute CVM measurements, allowing the end user to trust only the silicon rather than the inherently untrusted cloud provider. Deployments based on nested VM technology (nested VM-based and CoCo nested VMs) offer this capability since the parent VM functions as the hypervisor. This setup enables users to control the stack configuration for deploying child VMs, particularly the VM firmware code, which directly impacts the CVM measurements. Currently, instructions and firmware code for AMD SEV-SNP CVMs are available only on AWS, making such deployments a viable option for enhancing security in cloud environments where measurement reproducibility is otherwise impossible. However, we successfully deployed nested VMs only on Azure, specifically on child-capable VMs supporting AMD SEV-SNP.

Finally, we consider a man-in-the-middle attack scenario in which a malicious user deploys a non-Confidential VM between the root of trust (Trustee or SPIRE server) and the CVM. This intermediary VM runs a tampered agent that falsely requests an SVID/key on behalf of a CVM, while also presenting itself as the root of trust to the actual CVM. If successful, the attacker could obtain secrets for confidential workloads and access protected resources such as root file system decryption keys.

In SPIRE deployments, this attack is mitigated by embedding the SPIRE agent configuration, including the SPIRE server endpoint and TLS certificates, into the CVM's initrd. In a CoCo deployment, the KBS address, certificates, and policies are passed to the initdata and, thus, measured. Any change to this information would modify the CVM measurements in both SPIRE and CoCo deployments. Since the root of trust validates this measurement as part of the attestation process, it would reject the corrupted agent, effectively preventing the attack [31].

5.2. Performance Analysis

Deployment strategies differ mainly in how CVMs protect workloads. In Confidential node-driven deployments, worker nodes run in CVMs, indirectly securing workloads. In Confidential workload-driven deployments, workloads run directly in PodVMs, eliminating the need for worker nodes. Here, the operator can forgo a worker by untainting the master node, as all workloads run in individual PodVMs. We configured all pods with 8 GB of memory and 2 vCPUs, reflecting the smallest CVM flavor supported by cloud providers that is compatible with CoCo Peer Pods. Thus, independently of the deployment model, all workloads have the same resource capacity. Storage is provided through device-enabled volumes mounted in all deployments. Notice that we only experiment with the nested VMs model with the AMD SEV-SNP TEE, as this feature is currently supported exclusively on this TEE. For Intel TDX, we only experimented with CVM and Peer Pods because CSPs do not implement TD partitioning, which would enable the creation of nested VMs with TDX. Table 2 outlines the master's and workers' VM configurations for each strategy.

Table 2. Master’s and Workers’ VM configurations.

Model	TEE	Master	Worker
CVM	AMD SEV-SNP Intel TDX	2 vCPUs, 8 GB RAM	4 vCPUs, 16 GB RAM
Nested VM	AMD SEV-SNP	2 vCPUs, 8 GB RAM	4 vCPUs, 16 GB RAM
CoCo Bare Metal	AMD SEV-SNP	32 vCPUs, 128 GB RAM	none—untainted master
CoCo nested VMs	AMD SEV-SNP	8 vCPUs, 32 GB RAM	none—untainted master
CoCo Peer Pods	AMD SEV-SNP Intel TDX	4 vCPUs, 8 GB RAM	none—untainted master

CPU, Memory, and I/O Performance. We used the *sysbench* Unix module to assess the performance of each deployment in terms of CPU, RAM, and file I/O. For each resource benchmark, *sysbench* was configured to execute with 2 threads, as each workload was allocated in pods with 2 vCPUs. The file I/O measurement used a combined random read/write approach to emulate realistic mixed-access patterns. We ran *sysbench* for each resource 30 times to reduce variability and obtain statistically meaningful results. Figure 9 shows the confidence interval over the mean values for CPU, RAM, and I/O, with a confidence level of 95%.

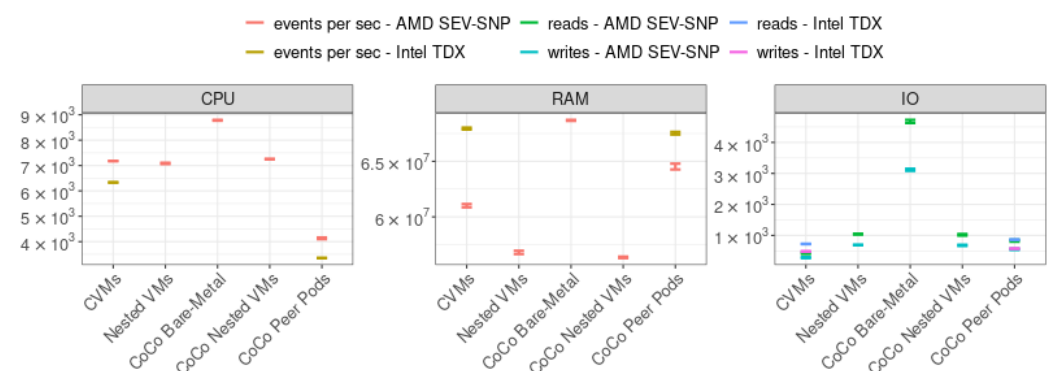
**Figure 9.** Performance of CPU, RAM, and IO, over the different K8s deployments on bare metal and Azure.

Figure 9 reveals that AMD SEV-SNP deployments exhibited comparable CPU performance across CVM, nested VM, and CoCo nested VM strategies, with throughput ranging between 7000 and 7500 events per second. In contrast, CoCo Bare Metal demonstrated superior performance, achieving an average of 8750 events per second. This improvement stems from its higher CPU frequency (3.1–3.91 GHz) compared to cloud CVMs (2.1–2.44 GHz). Notably, CoCo Peer Pods showed significantly reduced performance—approximately half that of other cloud CVM-based strategies. This degradation occurs because one vCPU is reserved for OS and CoCo component management, consequently limiting resources available for workloads.

Our comparison of Intel TDX and AMD SEV-SNP deployments reveals minimal performance differences between CoCo Peer Pods and CVMs, attributable to their respective clock speeds (2.44 GHz for SEV-SNP versus 2.1 GHz for TDX). Memory performance experiments demonstrate TDX’s slight advantage over SEV-SNP, likely stemming from their different memory protection architectures. SEV-SNP’s performance overhead arises from its Reverse Map Table mechanism, which continuously verifies write access to memory pages to prevent replay attacks [15]. In contrast, TDX employs a more efficient Physical Address Metadata Table (PAMT), which tracks page mappings and security states (e.g.,

encryption status), eliminating the need for constant validation [32]. These findings align with prior experimental studies [33].

For I/O performance, all deployment models followed similar patterns regardless of TEE technology. The superior I/O performance of CoCo Bare Metal stems from its use of local SSD, whereas in cloud deployments, the SSD volumes are attached to workloads through the network, affecting their performance.

Workload Initialization. We also measured workload initialization times for each deployment by configuring jobs with both the SPIRE agent and the workload itself. In node-driven deployments, a single SPIRE agent serves all workloads in a worker node, while in workload-driven deployments, each pod includes its own agent. As a result, workload initialization involves the following:

1. **[Workload-Driven:]** Initializing a CVM (PodVM).
2. **[Workload-Driven:]** Starting the SPIRE agent and performing node attestation with the SPIRE server.
3. Starting the workload, which will perform workload attestation to obtain its SVID, and then terminate execution.

We measured these steps using the Kubernetes `describe` command, which logs workload execution time from initialization. Table 3 presents the mean initialization time and standard deviation over 30 executions for each deployment.

Table 3. Pod initialization times.

Strategy	Mean	S.D.
CVM-based	<1 s	-
Nested VM-based	<1 s	-
CoCo Bare Metal	40.56 s	12.1%
CoCo nested VMs	39.8 s	5.2%
CoCo Peer Pods	92.03 s	16.2%

The first conclusion one may draw from Table 3 is that node-driven deployments initialize in less than one second—notice that the Kubernetes `describe` command has a precision of seconds and reports 0 for workloads lasting less than 1 s. This speed is due to the absence of VM and SPIRE agent initialization—since a single SPIRE agent serves all workloads in node-driven deployments, we consider it to be ready and running. In contrast, workload-driven deployments require significantly more time to initialize—ranging from approximately 40 to 92 s. CoCo Peer Pods take even longer, primarily due to the overhead associated with downloading and configuring PodVM components. Figure 10 breaks down the initialization times for workload-driven strategies, dividing the total time into VM boot and container startup phases.

Figure 10 shows that CoCo Bare Metal and CoCo nested VMs exhibit similar boot and container initialization times. In contrast, CoCo Peer Pods show a much higher variability in pod deployment times (refer to Table 3). Our analysis revealed that the initial pod deployments take longer, averaging around 86 s. However, after a few pods have been initialized, the time to deploy a pod is significantly reduced. The causes of this phenomenon could be multiple and are difficult to find once the pod does not provide boot logs. The most likely explanation involves warm-up effects and caching; the first PodVM could trigger the initial configuration of the host to boot the pod. The reason for the variability is that, for every new host, this setup must be performed, and once the host is prepared, the new pod can use this environment to boot faster.

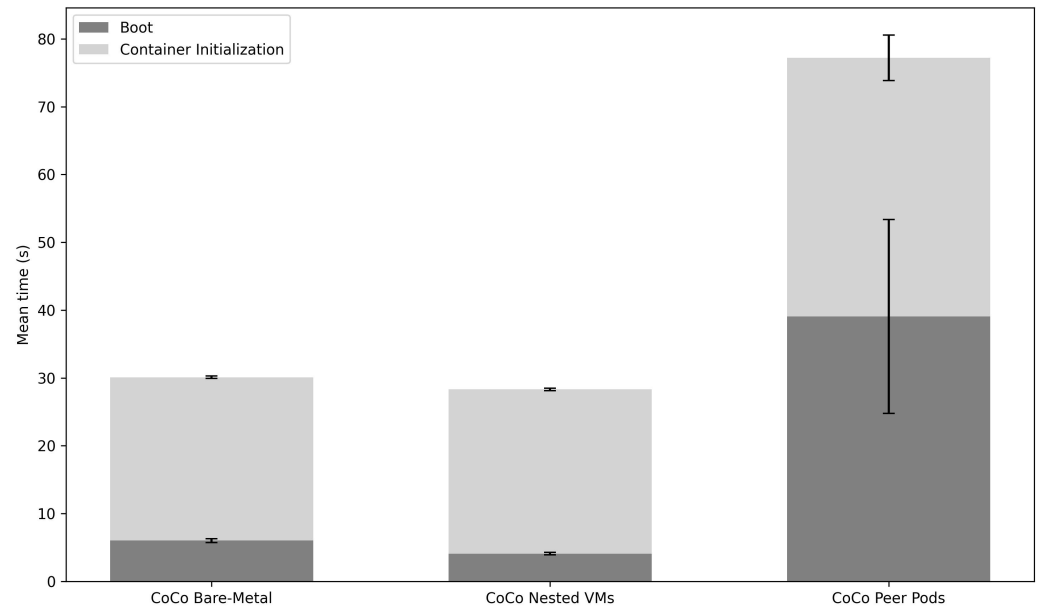


Figure 10. Phases of pod initialization for workload-driven deployments.

5.3. Financial Analysis

We analyzed the financial costs of different deployments. Our analysis focuses on answering the following question: **“What is the cost of hosting a confidential Kubernetes cluster with varying numbers of workloads across different deployment models?”**

For this financial analysis, we collected pricing data for AMD SEV-SNP and Intel TDX CVMs straight from AWS, Azure, and GCP portals. Since VM flavors can have different prices across regions, we selected the region with the lowest rate for each flavor. In addition, as prices fluctuate over time, we provide the collected data in Table A1 in Appendix A.

To ensure a fair comparison between Peer Pods and other deployment models, we considered K8s clusters without overcommitting. For all deployments, we assume that a VM with 8 vCPUs and 32 GB RAM is enough to host a K8s master node (the RedHat documentation recommends a master with 4 vCPUs and 19 GB of RAM to host 2000 pods [34]). We consider each worker node to consume 1 vCPU and 2 GB RAM to operate. Finally, different from the experiments performed in Section 5.2, here we consider a workload with requirements of 1 vCPU and 2 GB of RAM.

We calculate the hourly cost of a Kubernetes cluster deployed as Peer Pods by summing the cost of the master node VM with the most cost-effective CVM flavor that meets workload requirements. For deployments supporting neighboring pods within a single VM, we optimize costs by selecting the most powerful available VM flavors, thereby minimizing the percentage of vCPUs dedicated to worker node overhead. This approach reflects real-world deployment scenarios where workload variability leads to natural resource fragmentation across nodes. Note that advanced optimization techniques like workload scheduling and host consolidation fall outside this paper’s scope.

To determine the hourly cost of a Kubernetes cluster, we introduce the following notations. Let:

- $\mathcal{T}$ be the set of TEEs;
- $\mathcal{C}$ be the set of CSPs;
- $\mathcal{F}_c^t$ denote the set of VM flavors available in CSP c that support TEE t ;
- $\mathcal{F}_c^\emptyset$ denote the set of VM flavors in CSP c without TEE support.

Further, we define the following functions:

- $v : \mathcal{F}_c^t \rightarrow \mathbb{N}$, returning the number of vCPUs for a given flavor $f \in \mathcal{F}_c^t$;

- $\pi : \mathcal{F}_c^t \rightarrow \mathbb{R}$, returning the hourly price of a flavor $f \in \mathcal{F}_c^t$;
- $\mu : \mathcal{C} \rightarrow \mathcal{F}_c^\emptyset$, returning the VM flavor in CSP c that satisfies the master node constraints.

Let $\omega \in \mathbb{N}$ represent the maximum number of workloads to be hosted in the cluster. We then define κ , the function computing the minimum hourly cost of running a confidential Kubernetes cluster at CSP c , protected by TEE t , with capacity for ω workloads as follows:

$$\kappa(\omega, t, c) = \begin{cases} \pi(\mu(c)) + \left(\min_{f \in \mathcal{F}_c^t} \pi(f) \right) \cdot \omega, & \text{for Peer Pods deployments,} \\ \pi(\mu(c)) + \min_{f \in \mathcal{F}_c^t} \left(\left\lceil \frac{\omega}{v(f)-1} \right\rceil \cdot \pi(f) \right), & \text{otherwise.} \end{cases}$$

In short, $\kappa(\omega, t, c)$ sums the cost of the master node with the lowest-cost way to accommodate ω workloads, either one per VM (Peer Pods) or packed into multiple VMs (standard worker nodes). According to this methodology, we computed the hourly cost for different workload-hosting capacities across different CSPs and with various TEEs and present these results in Table 4.

Table 4. Hourly financial costs of Confidential K8s clusters under different deployments and workload (WL) levels.

Deployment	TEE	CSP	10 WL	100 WL	1000 WL
CVMs	AMD SEV-SNP	AWS	\$0.96	\$5.11	\$44.51
CVMs	AMD SEV-SNP	Azure	\$0.97	\$5.10	\$44.31
CVMs	Intel TDX	Azure	\$1.05	\$5.66	\$49.43
CVMs	AMD SEV-SNP	GCP	\$0.94	\$5	\$43.52
CVMs	Intel TDX	GCP	\$1.07	\$5.81	\$53.49
Nested VMs CoCo nested VMs	AMD SEV-SNP	Azure	\$0.97	\$5.10	\$44.31
Peer Pods	AMD SEV-SNP	AWS	\$1.13	\$8.87	\$86.27
Peer Pods	AMD SEV-SNP	Azure	\$1.14	\$8.88	\$86.28
Peer Pods	Intel TDX	Azure	\$1.24	\$9.88	\$96.28
Peer Pods	AMD SEV-SNP	GCP	\$1.12	\$8.77	\$85.27
Peer Pods	Intel TDX	GCP	\$2.29	\$20.47	\$202.27

First, it is important to note that AWS does not offer CVMs with Intel TDX support, while only Azure supports nested VMs, limited to AMD SEV-SNP technology. Across all cloud providers, AMD SEV-SNP VMs are more cost-effective than Intel TDX VMs. The results also indicate that CVMs and nested VMs have lower costs compared to Peer Pods, with AMD SEV-SNP on GCP being the most economical option at scale (\$43.52 for 1000 workloads). In contrast, Peer Pods using Intel TDX on GCP incur the highest costs (\$202.27 for 1000 workloads). This is primarily due to the smallest available Intel TDX VM instance on GCP requiring a minimum of 4 vCPUs, effectively doubling the cost compared to a 2 vCPU instance, which would have sufficed for the given workload requirements.

6. Related Works

In this section, we review the existing body of knowledge closely related to our paper.

Confidential Kubernetes. Constellation [35] reports a Confidential Kubernetes service in which every Kubernetes node (including the Kubelet, container runtime, and system components) is deployed in a CVM, thereby indirectly safeguarding all workloads scheduled on it. In addition, they encrypt the communication between nodes, encrypt storage, and provide means for remote attestation. The cryptographic keys required to access the network and the storage are provided only once a node passes the attestation. Similarly, Azure [36] and GCP [37] provide CSP-managed Confidential Kubernetes solutions, where worker nodes run in CVMs. In summary, all these approaches follow a confidential node-driven deployment strategy.

Confidential Containers. The Cloud Native Computing Foundation (CNCF) introduces three different possibilities to realize confidential containers, namely, container-centric, pod-centric, and node-centric [38]. They discuss the trade-off among these approaches in terms of security and efficiency. However, they do not define concrete deployment models like we do in this paper. Falcão et al. [11] propose a method for attesting container-centric workloads using SPIRE, achieved by developing an SGX workload attester that verifies the confidentiality and integrity of SCONe workloads running on Kubernetes clusters with SGX-supported nodes. Carlos et al. [7] showed that Confidential Containers (CoCo) incur excessively high setup latency for running serverless functions. TeeMate [8] proposes reducing the setup latency of enclave-based containers by sharing enclaves among multiple containers. Trusted Container Extensions (TCX) [9] are similar to Confidential Containers in the sense that they rely on Kata Containers to run workloads confidentially. TCX reports a low-performance overhead, but its applicability is restricted to AMD SEV VMs, while CoCo and SPIRE integrate with multiple TEEs. Azure Container Instances, based on the Parma [10] architecture, are reported to have less than one percent performance overhead compared with AMD SEV-SNP. Enriquillo et al. [39] propose securing CoCo containers by splitting its control interface into host-side and owner-side parts. Jana et al. [40] compare CVM offerings from major cloud providers. However, for containers, they only analyze the capabilities and shortcomings of Azure's offering.

7. Conclusions

Outsourcing computation to the cloud in the form of containerized microservice workloads is a popular trend. Confidential computing has the potential to make such outsourcing safer than ever before. In this work, we show how confidential Kubernetes can be verified despite the existing challenges. We also design, analyze, and evaluate five different Kubernetes deployment models on the infrastructure of three popular cloud providers with CPUs from two major vendors. Our deployment models use open-source tools for attestation, CoCo, and SPIRE, reducing the dependence on services specific to a cloud provider.

Our evaluation shows that performance remains similar across CPU vendors or cloud providers but can vary significantly across deployment models. We show that *nested VM* deployment models are the best in terms of security, performance, and cost-effectiveness. We also show that all deployment models perform similarly, except for the *CoCo Peer Pods* model, which we found to be the worst in terms of performance. In this work, we use CoCo and SPIRE for remote attestation and describe how they can be employed to verify the integrity of confidential Kubernetes deployments through CVMs.

Author Contributions: E.F. and A.B. are the main contributors to the conception of this work. F.S., C.P. and A.M. deployed and experimented with the Confidential Kubernetes cluster using different

strategies. E.F., A.S.M.A. and A.B. are the main authors of this manuscript. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded through a collaboration between Hewlett Packard Enterprise Brazil and the EMBRAP II unit UFCG-CEEI (Universidade Federal de Campina Grande), with incentive from the Informatics Law (Law No. 8.248, dated 23 October 1991).

Data Availability Statement: The performance experiments in this paper use only synthetic data produced by the stress tools and can be reproduced using the detailed parameters. The data used in the cost simulations are included in the appendix.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Appendix A

Table A1. Hourly VM costs by instance type and CSP (from portals, 21 March 2025).

CSP	TEE	Flavour Name	Child Capable	vCPUs	Memory	Price
Azure	AMD SEV-SNP	Standard_DC2as_v5	no	2	8 GB	\$0.086
Azure	AMD SEV-SNP	Standard_DC4as_v5	no	4	16 GB	\$0.172
Azure	AMD SEV-SNP	Standard_DC8as_v5	no	8	32 GB	\$0.344
Azure	AMD SEV-SNP	Standard_DC16as_v5	no	16	64 GB	\$0.688
Azure	AMD SEV-SNP	Standard_DC32as_v5	no	32	128 GB	\$1.376
Azure	AMD SEV-SNP	Standard_DC48as_v5	no	48	192 GB	\$2.064
Azure	AMD SEV-SNP	Standard_DC64as_v5	no	64	256 GB	\$2.752
Azure	AMD SEV-SNP	Standard_DC96as_v5	no	96	384 GB	\$4.128
Azure	Intel TDX	Standard_DC2es_v5	no	2	8 GB	\$0.096
Azure	Intel TDX	Standard_DC4es_v5	no	4	16 GB	\$0.192
Azure	Intel TDX	Standard_DC8es_v5	no	8	32 GB	\$0.384
Azure	Intel TDX	Standard_DC16es_v5	no	16	64 GB	\$0.768
Azure	Intel TDX	Standard_DC32es_v5	no	32	128 GB	\$1.536
Azure	Intel TDX	Standard_DC48es_v5	no	48	192 GB	\$2.304
Azure	Intel TDX	Standard_DC64es_v5	no	64	256 GB	\$3.072
Azure	Intel TDX	Standard_DC96es_v5	no	96	384 GB	\$4.608
Azure	AMD SEV-SNP	Standard_DC4as_cc_v5	yes	4	16 GB	\$0.172
Azure	AMD SEV-SNP	Standard_DC8as_cc_v5	yes	8	32 GB	\$0.344
Azure	AMD SEV-SNP	Standard_DC16as_cc_v5	yes	16	64 GB	\$0.688
Azure	AMD SEV-SNP	Standard_DC32as_cc_v5	yes	32	128 GB	\$1.376
Azure	AMD SEV-SNP	Standard_DC32as_cc_v5	yes	48	192 GB	\$2.472
Azure	AMD SEV-SNP	Standard_DC64as_cc_v5	yes	64	256 GB	\$3.296
Azure	None	Standard_D8ps_v6	no	8	32 GB	\$0.281
GCP	AMD SEV-SNP	n2d-standard-2	no	2	8 GB	\$0.085
GCP	AMD SEV-SNP	n2d-standard-4	no	4	16 GB	\$0.169

Table A1. Cont.

CSP	TEE	Flavour Name	Child Capable	vCPUs	Memory	Price
GCP	AMD SEV-SNP	n2d-standard-8	no	8	32 GB	\$0.338
GCP	AMD SEV-SNP	n2d-standard-16	no	16	64 GB	\$0.676
GCP	AMD SEV-SNP	n2d-standard-32	no	32	128 GB	\$1.352
GCP	AMD SEV-SNP	n2d-standard-48	no	48	192 GB	\$2.028
GCP	AMD SEV-SNP	n2d-standard-64	no	64	256 GB	\$2.704
GCP	AMD SEV-SNP	n2d-standard-80	no	80	320 GB	\$3.380
GCP	AMD SEV-SNP	n2d-standard-96	no	96	384 GB	\$4.056
GCP	AMD SEV-SNP	n2d-standard-128	no	128	512 GB	\$5.408
GCP	AMD SEV-SNP	n2d-standard-224	no	224	896 GB	\$9.463
GCP	Intel TDX	c3-standard-4	no	4	16 GB	\$0.202
GCP	Intel TDX	c3-standard-8	no	8	32 GB	\$0.403
GCP	Intel TDX	c3-standard-22	no	22	88 GB	\$1.109
GCP	Intel TDX	c3-standard-44	no	44	176 GB	\$2.218
GCP	Intel TDX	c3-standard-88	no	88	352 GB	\$4.435
GCP	Intel TDX	c3-standard-176	no	176	704 GB	\$8.871
GCP	Intel TDX	c3-standard-192	no	192	768 GB	\$9.677
GCP	None	e2-standard-8	no	8	32 GB	\$0.268
AWS	AMD SEV-SNP	m6a.large	no	2	8 GB	\$0.0864
AWS	AMD SEV-SNP	m6a.xlarge	no	4	16 GB	\$0.173
AWS	AMD SEV-SNP	m6a.2xlarge	no	8	32 GB	\$0.346
AWS	AMD SEV-SNP	m6a.4xlarge	no	16	64 GB	\$0.691
AWS	AMD SEV-SNP	m6a.8xlarge	no	32	128 GB	\$1.382
AWS	AMD SEV-SNP	m6a.12xlarge	no	48	192 GB	\$2.074
AWS	AMD SEV-SNP	m6a.16xlarge	no	64	256 GB	\$2.765
AWS	AMD SEV-SNP	m6a.24xlarge	no	96	384 GB	\$4.147
AWS	AMD SEV-SNP	m6a.32xlarge	no	128	512 GB	\$5.530
AWS	AMD SEV-SNP	m6a.48xlarge	no	192	768 GB	\$8.294
AWS	AMD SEV-SNP	m6a.metal	no	192	768 GB	\$8.294
AWS	None	t4g.2xlarge	no	8	32 GB	\$0.268

References

1. Qian, L.; Luo, Z.; Du, Y.; Guo, L. Cloud computing: An overview. In Proceedings of the IEEE International Conference on Cloud Computing, Beijing, China, 1–4 December 2009; Springer: Berlin/Heidelberg, Germany, 2009; pp. 626–631.
2. Thönes, J. Microservices. *IEEE Softw.* **2015**, *32*, 116. [\[CrossRef\]](#)
3. Bentaleb, O.; Belloum, A.S.; Sebaa, A.; El-Maouhab, A. Containerization technologies: Taxonomies, applications and challenges. *J. Supercomput.* **2022**, *78*, 1144–1181. [\[CrossRef\]](#)
4. Kubernetes. 2019. Available online: <https://kubernetes.io/> (accessed on 13 September 2025).
5. Asadujjaman, A.; Kabir, M.E.; Purohit, H.; Majumdar, S.; Wang, L.; Jarraya, Y.; Pourzandi, M. 5gfiver: Functional integrity verification for 5G cloud-native network functions. In Proceedings of the 2022 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), Bangkok, Thailand, 13–16 December 2022; pp. 162–169.

6. Asadujjaman, A.; Pontes, D.; Falcao, E.; Brito, A. Securing Confidential VMs in Public Clouds. In Proceedings of the 2024 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), Abu Dhabi, United Arab Emirates, 9–11 December 2024; pp. 55–62.
7. Segarra, C.; Feldman-Fitzthum, T.; Buono, D.; Pietzuch, P. Serverless confidential containers: Challenges and opportunities. In Proceedings of the 2nd Workshop on Serverless Systems, Applications and Methodologies, Athens, Greece, 22 April 2024; pp. 32–40.
8. Lee, C.; Hur, J.; Lee, S.; Lee, B. TEEMATE: Fast and Efficient Confidential Container using Shared Enclave. *arXiv* **2024**, arXiv:2411.11423. [CrossRef]
9. Brasser, F.; Jauernig, P.; Pustelnik, F.; Sadeghi, A.R.; Stapf, E. Trusted container extensions for container-based confidential computing. *arXiv* **2022**, arXiv:2205.05747. [CrossRef]
10. Johnson, M.A.; Volos, S.; Gordon, K.; Allen, S.T.; Wintersteiger, C.M.; Clebsch, S.; Starks, J.; Costa, M. Confidential Container Groups: Implementing confidential computing on Azure container instances. *Queue* **2024**, 22, 57–86. [CrossRef]
11. Falcão, E.; Silva, M.; Luz, A.; Brito, A. Supporting Confidential Workloads in SPIRE. In Proceedings of the 2022 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), Bangkok, Thailand, 13–16 December 2022; pp. 186–193. [CrossRef]
12. Intel Trust Domain Extensions. 2023. Available online: <https://www.intel.com/content/dam/develop/external/us/en/documents/tdx-whitepaper-final9-17.pdf> (accessed on 24 September 2024).
13. Intel. Intel TDX Virtual Firmware Design Guide. 2023. Available online: <https://cdrdv2-public.intel.com/733585/tdx-virtual-firmware-design-guide-rev-004-20231206.pdf> (accessed on 13 September 2025).
14. Intel Corporation. Intel TDX Module v1.5: TD Partitioning Architecture Specification. 2024. Available online: <https://www.intel.com/content/www/us/en/content-details/773039/intel-tdx-module-v1-5-td-partitioning-architecture-specification.html> (accessed on 13 September 2025).
15. AMD. AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More; Technical Report; AMD: Santa Clara, CA, USA, 2020.
16. Hunt, G.D.H.; Pai, R.; Le, M.V.; Jamjoom, H.; Bhattiprolu, S.; Boivie, R.; Dufour, L.; Frey, B.; Kapur, M.; Goldman, K.A.; et al. Confidential computing for OpenPOWER. In Proceedings of the Sixteenth European Conference on Computer Systems, Edinburgh, UK, 26–28 April 2021; EuroSys '21, pp. 294–310. [CrossRef]
17. Arm. Arm Confidential Compute Architecture. 2025. Available online: <https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture> (accessed on 13 September 2025).
18. Cloud, A. OpenAnolis Officially Launches Its First CSV Confidential Container Solution with HYGON. 2025. Available online: https://www.alibabacloud.com/blog/openanolis-officially-launches-its-first-csv-confidential-container-solution-with-hygong_599143 (accessed on 13 September 2025).
19. Randazzo, A.; Tinnirello, I. Kata Containers: An Emerging Architecture for Enabling MEC Services in Fast and Secure Way. In Proceedings of the 2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS), Granada, Spain, 22–25 October 2019; pp. 209–214. [CrossRef]
20. Kata Containers Contributors. The Kata Containers GitHub Project. 2025. Available online: <https://github.com/kata-containers/> (accessed on 13 September 2025).
21. Confidential Containers Project. Confidential Containers (CoCo). 2025. Available online: <https://github.com/confidential-containers> (accessed on 1 April 2025).
22. Confidential Containers Community. Cloud API Adaptor. 2023. Available online: <https://github.com/confidential-containers/cloud-api-adaptor> (accessed on 1 March 2024).
23. Feldman, D.; Fox, E.; Gilman, E.; Haken, I.; Kautz, F.; Khan, U.; Lambrecht, M.; Lum, B.; Fayó, A.M.; Nesterov, E.; et al. *Solving the Bottom Turtle: A SPIFFE Way to Establish Trust in Your Infrastructure via Universal Identity*; Book Sprints: Auckland, New Zealand, 2020.
24. Chandramouli, R.; Butcher, Z. A Zero-Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments. NIST SP 800-207A. 2023. Available online: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207A.pdf> (accessed on 17 August 2025). [CrossRef]
25. SPIFFE Contributors. The SPIFFE Runtime Environment GitHub Project. 2025. Available online: <https://github.com/spiffe/spire> (accessed on 13 September 2025).
26. Birkholz, H.; Thaler, D.; Richardson, M.; Smith, N.; Pan, W. RFC 9334: Remote ATtestation procedureS (RATS) Architecture; Internet Engineering Task Force (IETF): Fremont, CA, USA, 2023.
27. Community, C.C. Trusted Components for Attestation and Secret Management. 2023. Available online: <https://github.com/confidential-containers/trustee/> (accessed on 18 June 2024).
28. Contributors, C.C. Confidential Containers Attestation. 2025. Available online: <https://confidentialcontainers.org/docs/attestation/> (accessed on 13 September 2025).

29. Pontes, D.; Silva, F.; Falcão, E.; Brito, A. Attesting AMD SEV-SNP Virtual Machines with SPIRE. In Proceedings of the 12th Latin-American Symposium on Dependable and Secure Computing, La Paz, Bolivia, 16–18 October 2023; LADC '23, pp. 1–10. [CrossRef]
30. Pontes, D.; Silva, F.; Melo, A.; Asadujjaman, A.; Falcão, E.; Brito, A.; Filho, C. Multi-Platform and Vault-Free Attestation of Confidential VMs. In Proceedings of the 13th Latin-American Symposium on Dependable and Secure Computing, Recife, Brazil, 26–29 November 2024; LADC '24; pp. 241–251. [CrossRef]
31. Pontes, D.; Silva, F.; Melo, A.; Falcão, E.; Brito, A. Interoperable node integrity verification for confidential machines based on AMD SEV-SNP. *J. Internet Serv. Appl.* **2024**, *15*, 179–193. [CrossRef]
32. Intel. *Intel® Trust Domain Extensions (Intel® TDX)*; Technical Report; Intel: Santa Clara, CA, USA, 2023.
33. Coppolino, L.; D'Antonio, S.; Iasio, D.; Mazzeo, G.; Romano, L. An Experimental Evaluation of TEE technology Evolution: Benchmarking Transparent Approaches based on SGX, SEV, and TDX. *arXiv* **2024**, arXiv:2408.00443. [CrossRef]
34. Hat, R. Chapter 2. System and Environment Requirements. 2025. Available online: https://docs.redhat.com/en/documentation/openshift_container_platform/3.11/html/installing_clusters/install-config-install-prerequisites#system-requirements (accessed on 13 September 2025).
35. Constellation. Confidential Kubernetes. 2025. Available online: <https://constellation-docs.netlify.app/constellation/overview/confidential-kubernetes> (accessed on 13 September 2025).
36. Azure, M. Deploy an AKS Cluster with Confidential Containers and an Automatically Generated Policy. 2025. Available online: <https://learn.microsoft.com/en-us/azure/aks/deploy-confidential-containers-default-policy> (accessed on 13 September 2025).
37. Platform, G.C. Encrypt Workload Data In-Use with Confidential GKE Nodes. 2025. Available online: <https://cloud.google.com/kubernetes-engine/docs/how-to/confidential-gke-nodes> (accessed on 13 September 2025).
38. CNCF. Confidential Containers: Design Overview. 2025. Available online: <https://confidentialcontainers.org/docs/architecture/design-overview/> (accessed on 13 September 2025).
39. Valdez, E.; Ahmed, S.; Gu, Z.; De Dinechin, C.; Cheng, P.C.; Jamjoom, H. Crossing Shifted Moats: Replacing Old Bridges with New Tunnels to Confidential Containers. In Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, Salt Lake City, UT, USA, 14–18 October 2024; pp. 1390–1404.
40. Eisoldt, J.; Galanou, A.; Ruzhanskiy, A.; Küchenmeister, N.; Baburkin, Y.; Dai, T.; Gudymenko, I.; Köpsell, S.; Kapitza, R. SoK: A cloudy view on trust relationships of CVMs—How Confidential Virtual Machines are falling short in Public Cloud. *arXiv* **2025**, arXiv:2503.08256.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.